Python Engineering - Class 09 Reference Guide

Modulos e imports

Guía de consulta de sintaxis y herramientas utilizadas en esta clase. Incluye qué hace cada elemento, cómo se escribe, qué recibe o produce y cuándo conviene utilizarlo.

Mapa rápido

Concepto	Sintaxis	Idea
Módulo	.py file	Archivo .py reutilizable que puede exponer funciones, clases o constantes.
import	import module	Carga un módulo y permite acceder a sus nombres mediante module.name.
from ... import ...	from module import f	Importa nombres concretos de un módulo al namespace actual.
__name__	special module variable	Variable especial que permite distinguir ejecución directa de importación.
main()	def main():	Función convencional que orquesta la ejecución principal de la aplicación.
Main guard	if __name__=='__main__':	Evita que main() se ejecute automáticamente cuando el archivo se importa como módulo
Dependencias	import numpy as np	Cada módulo debe importar directamente las dependencias que utiliza.
Separación de responsabilidades	physics / processing / orchestration	Divide física, procesamiento y orquestación en módulos con responsabilidades distintas.

1. Módulo

Sintaxis: `.py file`

Archivo .py reutilizable que puede exponer funciones, clases o constantes.

```
# vertical_motion.py

def euler_step(...):
    ...
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

2. import

Sintaxis: `import module`

Carga un módulo y permite acceder a sus nombres mediante module.name.

```
import numpy as np
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

3. from ... import ...

Sintaxis: `from module import f`

Importa nombres concretos de un módulo al namespace actual.

```
from vertical_motion import euler_step
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

4. __name__

Sintaxis: `special module variable`

Variable especial que permite distinguir ejecución directa de importación.

```
print(__name__)
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

5. main()

Sintaxis: `def main():`

Función convencional que orquesta la ejecución principal de la aplicación.

```
def main():
    ...
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

6. Main guard

Sintaxis: `if __name__=='__main__':`

Evita que main() se ejecute automáticamente cuando el archivo se importa como módulo.

```
if __name__ == '__main__':
    main()
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

7. Dependencias

Sintaxis: `import numpy as np`

Cada módulo debe importar directamente las dependencias que utiliza.

```
# Cada módulo importa directamente lo que utiliza
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

8. Separación de responsabilidades

Sintaxis: `physics / processing / orchestration`

Divide física, procesamiento y orquestación en módulos con responsabilidades distintas.

```
vertical_motion.py -> física
data_processing.py -> análisis
main.py -> orquestación
```

Pregunta de control: antes de ejecutar el ejemplo, predice el tipo, el valor aproximado y la shape cuando corresponda.

Errores comunes

- Confundir el nombre de una función con su llamada usando paréntesis.
- No revisar tipos, índices o shapes antes de una operación.
- Usar una herramienta porque funciona sin poder explicar qué recibe y qué devuelve.

Chuleta mental

```
Módulo: .py file
import: import module
from ... import ...: from module import f
__name__: special module variable
main(): def main():
Main guard: if __name__=='__main__':
Dependencias: import numpy as np
Separación de responsabilidades: physics / processing / orchestration
```